朱雀 JavaScript 引擎 0.6 版发布

https://github.com/nupha/zuqe

自 0.5 发布以来，倏忽月余。原以为只是修修补补的小版本，不成想越捯饬越多，改到后来已面目全非，索性便作一个大版本罢。

Zuqe 引擎从第一天起就是纯同步跑的：JS 代码一行一行走，Promise 挂在那等回调，事情倒是能做完，只是跟 Rust 世界的 async/await 半点儿挨不上，心中颇痒。若想写个 HTTP 服务、做个 WebSocket 连接，嵌入zuqe后两边各说各话，着实头疼。此次 r0.6 的核心心思，便是把这条路打通。

先说最要紧的：r0.6 新添了一个 `async_support` feature，把 JS 的 Promise 和 Rust 的 async/await 连接起来。JS 端的 Promise 可以转成 Rust 的 `Future`，丢到 tokio 里一起跑；反过来，引擎本身也包装成了 `Future`，`Runtime::run_async()`。

写法倒也简单，大概如此：

```
// 假设有一个 promise: Promise 对象
// 将 Promise 转 Future
let promise_fut = promise.into_future(ctx);
// 然后可以融入rust异步编程体系，比如这样
let r = promise_fut.await;
```

明白人一眼便知，不过是牵了根线，让两边的异步世界能彼此看见罢了。

此版本中，Promise 还多了几种创建方式。以前想在 Rust 里弄个 Promise，得经过 `Promise::crate(ctx)`，如今可以用通信通道(channel)，或者数据槽位来创建。这样甚至能实现跨线程数据推送。具体做法有四种：

- **create_with_job**：每 tick 轮询一次，数据到了就 resolve，没到就等下一轮。最适合做异步迭代器的数据源。
- **create_with_receiver**：从 channel 收一条消息，收了就 resolve。跨线程单次通信的轻量选择。
- **create_with_channel**：返回 `(Promise, Sender)` 一对，自己发消息来 resolve。最灵活。
- **create_with_result_slot**：单次提交，不用 channel，适合简单的传值场景。

记得早些时候看 spec 里 Promise 的各种用法，总觉得那 resolving functions 绕来绕去，不如直接 `promise.resolve(value)` 来得爽利。这次也一并改了，`Promise::resolve()` 和

`Promise::reject()` 都公开为公共 API，用起来干净许多。

以 channel 为例，创建一个 Promise 再拿个发送端，另一个线程算完结果发过来即可：

```
use zuqe::builtins::promise::Promise;

// 创建 Promise，拿一个发送端。假设提交的数据为i32类型
let (promise, tx) = Promise::create_with_channel::<i32>(ctx, |_, n: i32| {
    Ok(JSValue::Int(n))
})?;

std::thread::spawn(move || {
    tx.send(42).unwrap();  // 另一个线程推送结果，Promise 即 resolve
});
```

若不想引入 channel，也有更轻量的 slot 方式——创建一个数据槽位，另一个线程把结果往里一塞就完事：

```
let slot = PromiseResultSlot::<i32>::new();
let promise = Promise::create_with_result_slot(ctx, slot.clone(), |_, n: i32| {
    Ok(JSValue::Int(n))
})?;

std::thread::spawn(move || {
    slot.commit(42).unwrap();  // 值进去了，Promise 自动 resolve
});
```

两种方式各有用处，channel 适合多条数据、跨线程多次通信的场景；slot 更轻量，单次提交即可，各自取舍便是。

说完 Promise，再看**异步迭代器**。`for await...of` 这套语法在 r0.5 时只是个壳子，如今真正能跑了。Rust 这边可经过 channel 或者 slot 往 JS 世界推送数据，JS 那边就像处理流一样，一条一条收。WebSocket 消息来了、文件按行异步读出来、数据库游标数据一条一条推送... 这种场景用起来，倒真有几分顺手。

以 channel 为例：

```rust
use zuqe::builtins::iterator::AsyncIterableIterator;

// 创建 channel 驱动的异步迭代器，返回 (iter, sender) 对
// iter 在 JS 那边可以用 for await...of
// tx 这个发送端，可以在 rust 里向 js 推数据
let (iter, tx) = AsyncIterableIterator::<Reply>::create_with_channel(
    ctx,
    |cx, msg: Reply| {
        convert_reply_to_jsvalue(cx, msg) // 将rust数据转为JSValue
    }
)?;

std::thread::spawn(move || {
    tx.send(msg1).unwrap();
    tx.send(msg2).unwrap();
    drop(tx);  // 数据发完，断开 channel
});
```

slot 方式也类似：

```rust
let data_slot = PromiseResultSlot::<Reply>::new();
let slot = data_slot.clone();

std::thread::spawn(move || {
    // 另一个线程产生数据
    match slot.commit(msg) {
        Ok(()) => { /* 提交成功 */ }
        Err(m) => { /* 上次数据还没消费，稍后再试 */ }
    }
});

let iter = AsyncIterableIterator::create_with_result_slot(
    ctx, data_slot,
    |cx, msg: Reply| {
        convert_reply_to_jsvalue(cx, msg)
    }
)?;
```

JS 那边便只是：

```
for await (const value of asyncIterator) {
  console.log(value);
}
```

除去异步这块大头（Promise <=> Future, AsyncIterableIterator），还有几处顺手完成的零碎修改：

- 增加了 `serde_json_support` feature，开启后 JSValue 和 `serde_json::Value` 可以互转。从 HTTP 接口拿到 JSON，直接塞给 JS 环境用；JS 算完结果，也能序列化成 JSON 输出。不做 fancy 的事，但确实实用。
- Array 在 Rust 侧多了几个工厂方法，`Array::create()` 、`Array::from_values()` 、`from_slice()` 、`with_capacity()`，创建数组方便了许多。原生模块要给 JS 返回一组数据，不必再走那些弯弯绕绕的路了。
- `JSValue` 上加了 `ensure_object()` 和 `ensure_function()` 方法，类型不对直接抛 TypeError。以前写参数校验要 `if let Some(obj) = val.as_object()` 再 else 里抛异常，啰嗦得紧，如今一行完事。
- 还有一个可选 feature `crossbeam_channel` —— 不开 feature 时用标准库的 `std::sync::mpsc`，够用但不算快；开了就改用 crossbeam channel，适合高并发场景，代价是多引入一个第三方 crate。

内部改动也不少，这里随手列几笔，不细说：

- **Promise 系统重构**：文件改名（resolving → settle、job → microtask），Job 从 FnOnce 改成 Fn 支持轮询，内部数据结构名也做了统一。改名这种活虽然不是功能增强，但对日后维护确有帮助。
- **TypedArray 构造函数重构**：之前从迭代器取值时逐个转类型，不符合 ECMAScript 规范。改成了先收齐再统一转。
- **GET_ARRAY_EL3 字节码**：读取数组元素从原来多条指令缩成一条，算是个小优化。
- **Atom 系统**：`Atom::new()` 改成 `Atom::from_str()`，名字一看便知是干什么的。
- **字符串系统**：移掉了热路径上一段调试断言代码，跑字符串操作时比以往快了那么一丁点。

回顾这 r0.6，本想着只是给 Promise 加个 async 通道，结果牵一发而动全身——微任务（microtask）队列要改、Job 系统要改、Runtime 要改、还要加异步迭代器。中间曾一度觉得，这到底是在做 JS 引擎还是在做 async runtime?

某遥拜顿首。